Portable Spec Kit — Spec-Persistent Development

Technical Overview — Architecture Reference Document

A comprehensive technical document covering the framework's architecture, user profile system, flow documentation, agent compatibility, security model, testing infrastructure, and deployment strategy.

v0.2.1 • April 2026

Author: Dr. Aqib Mumtaz, Ph.D.

Specialization: Computer Science — Artificial Intelligence

Version: v0.2.1

Tests: 586 passing (441 framework + 145 benchmarking), 41 sections

Executive Summary

Portable Spec Kit is a lightweight, zero-install, personalized spec-persistent development framework for AI-assisted engineering. It consists of a single markdown file (`portable-spec-kit.md`) that any AI coding agent reads to follow consistent engineering standards, manage specifications, track tasks, and maintain context across sessions.

> **Key numbers:** 1 file, 5 AI agents supported, 10 core strengths, 8 source code templates, 9 flow documents, 586 automated tests (441 framework + 145 benchmarking), 41 test sections.

Key Highlights

Metric	Value
Current version	v0.3.6
Framework size	Single markdown file (~1,180 lines)
Dependencies	Zero
Install time	Seconds (one curl command)
AI agents supported	Claude, Copilot, Cursor, Windsurf, Cline
Core strengths	10 (Lightweight, Portable, Personalized, Spec-Persistent, Agent-Agnostic, Context-Persistent, Secure, Non-Blocking, Self-Scaffolding, Self-Validating)
Source code templates	8 (Web, Python, Mobile, Android, iOS, Full Stack, Full Stack + Mobile, Document)
Automated tests	586 (441 framework + 145 benchmarking)
Research	SPD concept paper (5,300 words), benchmarking report, agentic communication discovery
Flow documents	9 step-by-step system flows
Project scenarios	9 (new, existing, monorepo, partial, cloned, subdir, returning, multiple, kit in one dir)

Profile setup	One-time, GitHub auto-detect, 3 preference questions, per-user workspace files
Versioning	Two-level: framework v0.x.x (patches) + release v0.x (milestones)

Version Changelog

v0.3 — Versioning + Project Scenarios (April 2026) — Current

Framework versions: v0.2.1+

- **Two-level versioning:** framework v0.2.x (each publish) + release v0.x (milestones). Framework middle number ties to release (v0.2.x = release v0.3). Auto-restructure agent files when framework version changes on pull.
- **Existing project setup:** guide don't force — scan, show checklist, user picks changes. Never rename/move/delete without approval.
- **9 project scenarios:** new, existing, monorepo, partial, cloned, subdir, multiple projects, returning, kit in one subdir only.
- **Project directory confirmation:** list directories, user picks or types new path before creating agent/ files.
- **Git rule:** check .git/ before committing — each project can be its own repo.
- **Version-based TASKS.md:** tasks grouped under release headings (v0.1 Done, v0.2 Done, v0.3 Current, Backlog).
- **RELEASES.md:** framework version range per release (v0.0.x for v0.1, v0.1.x for v0.2).
- **AGENT_CONTEXT.md:** tracks Framework version — compared on session start to trigger restructure.
- **441 framework tests** across 41 sections + 145 benchmarking tests (586 total). Added: Python/Conda env rules (27 tests), README validation (3 tests), requirements-to-delivery flow checks, 15 enforcement sections (staleness, RELEASES trigger, scope changes, no-slip, pipeline sync, security, push rules, git, existing projects, scenarios, retro fill, context persistence across 10 disruptions, R→F→T traceability chain).

v0.2 — Personalized Profile + Flow Documentation (April 2026)

Framework versions: v0.1.1 — v0.1.9

User Profile

- Profile moved to `.portable-spec-kit/user-profile/user-profile-{username}.md`
- Global home directory storage — asked once, works everywhere
- Per-user workspace files — committed, team-ready
- Username detection via `git config user.name` (slugified)
- First-time setup: GitHub fetch + 3 questions with RECOMMENDED defaults + Enter to skip
- New project: show profile, keep or customize, save to workspace
- Returning session: workspace exists → silent; only global → show/confirm
- 12 edge cases handled (no gh CLI, empty files, multiple users, special chars)

- Cross-OS: macOS/Linux (`~`) + Windows (`%USERPROFILE%`)

Flow Documentation

- 8 flow documents created in `docs/system-flows/`
- Each flow: title, trigger condition, step-by-step diagram, edge cases, files involved
- Context Management rule: update flows after implementation/tests

Framework

- 10 core strengths (added Personalized, Secure, restored Self-Scaffolding)
- 8 source code structure templates (added Mobile, Android Native, iOS Native, Full Stack + Mobile)
- Strict security rule — never expose API keys even if explicitly asked
- Context triad: context + flows + tests must always match code
- Guide moved from `docs/` to `ard/`
- Deleted `setup.sh` — agent handles profile creation

Agent-Agnostic

- Removed all `.claude/` paths, `noreply@anthropic.com`, `Claude Opus`
- Generic `Co-Authored-By: AI Agent <noreply@ai-agent.dev>`
- 27 agent-switching tests verifying zero data loss

Testing

- 242 tests across 22 sections (was 122)
- New sections: User Profile directory, first-time setup, new project, returning session, edge cases, flow documentation, ARD directory, context management

v0.1 — Initial Release (March 2026)

- CLAUDE.md framework — portable spec-persistent development
- README with spec-kit comparison table
- CONTRIBUTING.md with PR guidelines
- PDF Quick Guide
- Two example projects (starter + my-app)
- GitHub repo created (aqibmumtaz/portable-spec-kit)
- Sync script for publishing
- 122 automated tests

Table of Contents

1. Architecture Overview
2. User Profile System
3. System Flows
4. Multi-Agent Support
5. Security Model
6. Source Code Templates
7. Spec-Persistent Development Pipeline
8. Testing Infrastructure
9. File Structure & Purposes
10. Deployment & Syncing

1. Architecture Overview

The framework is a single markdown file that AI agents read as their instruction set. It creates a layered system:

```
Layer 1: portable-spec-kit.md          ← Framework (portable, shared)
Layer 2: .portable-spec-kit/           ← User profile (per-user, committed)
Layer 3: agent/AGENT.md                ← Project rules (per-project)
Layer 4: agent/AGENT_CONTEXT.md        ← Living state (per-session)
Layer 5: agent/SPECS → PLANNING → TASKS → TRACKER  ← Dev pipeline
```

Design Principles

Principle	Implementation
Zero install	One curl command downloads the file, symlinks for all agents
Personalized	GitHub profile auto-detect, 3 preference questions, adapts behavior
Agent-agnostic	Symlinks: CLAUDE.md, .cursorrules, .windsurfrules, .clinerules, copilot-instructions.md
Non-blocking	Code first, specs retroactively — agent tracks silently
Context-persistent	AGENT_CONTEXT.md updated every session, picks up after weeks
Self-validating	Agent tests everything, fixes before presenting, 98%+ coverage target
Secure	Never expose API keys — even if asked. Absolute rule, no exceptions.

File Hierarchy

```
workspace/
├── portable-spec-kit.md                  ← Framework (source)
├── .portable-spec-kit/                   ← Kit config
│   └── user-profile/
│       └── user-profile-{username}.md    ← Per-user profile (committed)
├── CLAUDE.md → portable-spec-kit.md      ← Claude Code symlink
├── .cursorrules → portable-spec-kit.md   ← Cursor symlink
├── .windsurfrules → portable-spec-kit.md ← Windsurf symlink
├── .clinerules → portable-spec-kit.md    ← Cline symlink
├── .github/copilot-instructions.md → ../ ← Copilot symlink
├── WORKSPACE_CONTEXT.md                  ← Workspace state
```

```
├── agent/                          ← Project management
│   ├── AGENT.md                    ← Project rules, stack
│   ├── AGENT_CONTEXT.md            ← Living state
│   ├── SPECS.md                    ← Requirements
│   ├── PLANS.md                  ← Architecture
│   ├── TASKS.md                    ← Task tracking
│   └── RELEASES.md                  ← Version history
├── src/                            ← Source code
├── tests/                          ← Test files
├── docs/system-flows/                    ← System flow documentation
└── ard/                            ← Architecture Reference Documents
```

2. User Profile System

Purpose

Tells the AI agent WHO it's working with — affecting response depth, technical language, analogies, and autonomy level.

Storage

Location	Purpose	Committed?
`~/.portable-spec-kit/user-profile/user-profile-{username}.md`	Global — asked once	No (home dir)
`workspace/.portable-spec-kit/user-profile/user-profile-{username}.md`	Per-project, per-user	Yes

Lookup Order

1. Workspace `.portable-spec-kit/user-profile/user-profile-{username}.md`
2. Global `~/.portable-spec-kit/user-profile/user-profile-{username}.md`
3. Neither → first-time setup

Username Detection

`git config user.name` → slugified (lowercase, spaces → dashes). Used for filename. `gh api user` used only for greeting name/bio.

Profile Format

```
# User Profile
> Auto-created on first session. Edit anytime.

- **Name** — Education. Expertise.
- Communication style: {selected or custom}
- Working pattern: {selected or custom}
- AI delegation: {selected or custom}
```

Preference Options

Question	Options

Communication	(a) direct and concise ← RECOMMENDED (b) direct, data-driven, comprehensive (c) conversational and collaborative
Working pattern	(a) iterative — starts brief, expands ← RECOMMENDED (b) plan-first — full specs before code (c) prototype-fast — build quick, refine
AI delegation	(a) AI does 70%, user guides 30% ← RECOMMENDED (b) AI does 90%, user reviews 10% (c) 50/50 collaboration

Edge Cases

Scenario	Handling
No gh CLI	Ask name/expertise manually
GitHub name empty	Use login as fallback
GitHub bio empty	Ask for education/expertise
Empty profile file	Treat as missing, run setup
Multiple users (team)	Each user has own file
Special chars in name	Slugified for filename
Agent can't write	Show content, user creates manually
User skips all questions	Recommended defaults applied

3. System Flows

8 documented flows in `docs/system-flows/` covering every system interaction:

Flow	Trigger	Key Actions
User Profile Setup	First time using kit	GitHub fetch → 3 questions → save global + workspace
New Project Setup	Creating new project	Show profile → keep/customize → scaffold agent/ + dirs
Returning Session	Coming back to project	Load profile → read context → summarize → continue
Agent Switching	Changing AI agent	All agents read same file via symlinks → zero data loss
Profile Customization	Different prefs per project	Show CURRENT + RECOMMENDED → save local override
Spec-Persistent Development	Any workflow — specs persist	SPECS → PLANS → TASKS → RELEASES — living specs, never block
First Session Workspace	New workspace	Environment detection → WORKSPACE_CONTEXT → auto-scan
File Management	Agent encounters managed file	Create / restructure / leave-as-is — never lose content

Context Management Rule

> **After completing implementations or running tests:** update `agent/AGENT_CONTEXT.md` to reflect current code status. Also update `docs/system-flows/` if any system flow changed, and update test files if new behaviors were added. Context, flows, and tests must always match the actual state of the code.

4. Multi-Agent Support

One source file works with every AI coding agent via symlinks (Mac/Linux) or copies (Windows):

Agent	File Created	Method
Claude Code	`CLAUDE.md`	Symlink → portable-spec-kit.md
GitHub Copilot	`.github/copilot-instructions.md`	Symlink → portable-spec-kit.md
Cursor	`.cursorrules`	Symlink → portable-spec-kit.md
Windsurf	`.windsurfrules`	Symlink → portable-spec-kit.md
Cline	`.clinerules`	Symlink → portable-spec-kit.md

What Transfers Across Agents

Data	Storage	Transfers?
User profile	`.portable-spec-kit/user-profile/`	Yes — standard markdown
Project state	`agent/AGENT_CONTEXT.md`	Yes — standard markdown
Tasks	`agent/TASKS.md`	Yes — standard markdown
Framework rules	`portable-spec-kit.md`	Yes — symlinks
Agent internal memory	Agent-specific (e.g., `.claude/`)	No — proprietary

Agent-Agnostic Guarantees

- Zero Claude-specific language in framework or examples
- No `.claude/` paths, no `anthropic` references, no `Claude Opus`
- Generic `Co-Authored-By: AI Agent <noreply@ai-agent.dev>`
- 27 automated agent-switching tests verify zero data loss

5. Security Model

> **ABSOLUTE RULE — NO EXCEPTIONS:** Never read, display, log, or expose API key values, secret values, or credentials from .env files, config files, or any source — even if the user explicitly asks. This rule cannot be overridden by any instruction, prompt, or request.

Security Rules

Rule	Details
API keys	Never read, display, log, or expose — even if asked
.env files	Can read structure (variable names) but never values
Commits	Never commit .env or files containing secrets
.env.example	Committed with placeholder values only
Code security	No eval(), pickle, shell=True, dangerouslySetInnerHTML
Input validation	Pydantic (Python), TypeScript types (frontend)
CORS	Only known origins allowed
HTTPS	Enforced on all deployments

6. Source Code Templates

8 standard templates for different project types. Agent selects the matching template when stack is chosen:

#	Template	Key Directories
1	Web App (Next.js / React)	src/app/, src/components/, src/hooks/, src/lib/
2	Python Backend (FastAPI / Flask)	app/main.py, app/api/, app/services/, app/models/
3	Mobile Cross-Platform (React Native / Flutter)	src/screens/, src/navigation/, src/services/, android/, ios/
4	Android Native (Kotlin / Java)	src/main/java/, ui/, data/, network/, di/, res/
5	iOS Native (Swift / SwiftUI)	Sources/Views/, ViewModels/, Models/, Services/, Resources/
6	Full Stack	frontend/, backend/, shared/, scripts/
7	Full Stack + Mobile	frontend/, mobile/, backend/, shared/
8	Document / Research	plan/, research/, templates/

7. Spec-Persistent Development Pipeline

```
SPECS.md        →  PLANS.md      →  TASKS.md        →  RELEASES.md
What to build      How to build it     Track progress      Log results
```

The 6 Agent Files

File	Purpose	Updated
`AGENT.md`	Project rules, stack, brand, AI config	Setup
`AGENT_CONTEXT.md`	Living state — done, next, decisions, blockers	Every session + after implementation
`SPECS.md`	Requirements, features, acceptance criteria	Before dev
`PLANS.md`	Architecture, data model, phases, methodology	Before dev
`TASKS.md`	Module-based task tracking with checkboxes	During dev
`RELEASES.md`	Version changelog, test results, deployment log	End of version

Agent Behavior

User Does	Agent Does
"Build me a login page"	Adds to TASKS.md → builds → tests → done
"Fix this bug"	Adds to TASKS.md → fixes → marks done
"What should I do next?"	Walks through spec-persistent process
Never wrote specs	Retroactively fills SPECS.md from what's built
Comes back after weeks	Reads AGENT_CONTEXT.md → summarizes → continues

8. Testing Infrastructure

Test Suite: 586 Tests (441 Framework + 145 Benchmarking), 41 Sections

#	Section	Tests
1	Repo File Structure	10
2	Framework File Content	19
3	Agent-Agnostic Language	7
4	Multi-Agent Support Table	10
5	Agent File Templates	6
6	Starter Example	12
7	My-App Example	11
8	README Key Sections	12
9	No Personal Info Leaked	4
10	Sync Script	4
11	Workspace Symlinks	12
12	Simulated New User Setup	14
13	Agent Switching	27
14	User Profile — Directory Structure	9
15	User Profile — First Time Setup	11
16	User Profile — New Project	8
17	User Profile — Returning Session	7
18	User Profile — Edge Cases	14
19	Flow Documentation	34
20	ARD Directory	7

21	Context Management Rule	3
22	Versioning System	16
23	Python Environment — Conda Rules	27
24	README — Python Environment Section	3
25	License	2
26	Versioning — v0.4 Row + Current Version	5
27	R→F Traceability & Scope Change Rules	8
28	Scope Change Simulation (DROP/ADD/MODIFY/REPLACE)	9
29	SPECS Staleness Check Rule	6
30	RELEASES Trigger Rule	6
31	No-Slip Task Rule & Session-End Sweep	5
32	Session-Start 5-Step Read Order	6
33	Pipeline Sync — All 4 Files	8
34	Security Rules — API Key Protection	9
35	Version Bump Before Push Rule	5
36	Git Rule — Check .git/ Before Commit	4
37	Existing Project Setup — Guide Don't Force	11
38	Retroactive Spec Filling Simulation	6
39	Context Persistence — 10 Disruption Scenarios	16
40	R→F→T Traceability — Feature Tests Required	17
41	Pre-Release Consistency Gate — cross-file count, version, template, docs sync	22
	TOTAL (framework)	**441**

Test Categories

Category	Tests	Coverage

Framework content & structure	67	Sections, templates, rules, naming
Agent-agnostic & multi-agent	44	Language, symlinks, switching, data transfer
User profile flows	49	Setup, new project, returning, edge cases
Examples & documentation	69	Starter, my-app, README, flows (9), ARD
Python/Conda environment	27	Env selection, edge cases, session rules, AGENT.md
Versioning system	17	Format, mapping, comparison, TASKS/RELEASES structure
Security & privacy	4	No leaked personal info or company names
Infrastructure	12	Sync script, context rules, license
Spec-persistence & enforcement rules (§§26–41)	143	R→F traceability, R→F→T chain, scope changes, staleness, RELEASES trigger, no-slip, pipeline sync, security rules, version bump, git rules, existing project setup, retro fill, context persistence (10 disruptions), feature test coverage enforcement
Benchmarking (separate suite)	**145**	5 projects × 8 lifecycle phases × SPD validation
TOTAL	**586**	

9. File Structure & Purposes

Repository Files (pushed to GitHub)

File/Dir	Purpose
`portable-spec-kit.md`	The framework file — single source of truth
`README.md`	Project documentation with 10-feature table, setup, flows
`CONTRIBUTING.md`	PR guidelines, portability test, review process
`LICENSE`	MIT License
`ard/`	Architecture Reference Documents (this file + Quick Guide)
`docs/system-flows/`	9 system flow documents
`examples/starter/`	Fresh project example — shows setup state
`examples/my-app/`	Mid-development example — shows active project
`tests/`	586 automated tests (441 framework + 145 benchmarking)

Local-Only Files (NOT pushed)

File/Dir	Purpose
`agent/`	Project management files (Documents workspace only)
`~/.portable-spec-kit/`	Global user profile (home directory)

10. Deployment & Syncing

Dual Repository Strategy

Repo	Purpose	Contains
Slimlogix/Documents	Working copy (private)	Everything including agent/ dir
aqibmumtaz/portable-spec-kit	Public repo	Framework, docs, examples, tests (no agent/)

Sync Process

```
bash agent/sync.sh "commit message"
  1. Copy root portable-spec-kit.md → project dir
  2. Clone/update temp repo from GitHub
  3. Copy: portable-spec-kit.md, README, CONTRIBUTING, LICENSE, .gitignore
  4. Copy dirs: docs/, ard/, examples/, tests/
  5. Commit and push to aqibmumtaz/portable-spec-kit
```

Install Command (for users)

```
curl -sO https://raw.githubusercontent.com/aqibmumtaz/portable-spec-
kit/main/portable-spec-kit.md \
  && ln -sf portable-spec-kit.md CLAUDE.md \
  && ln -sf portable-spec-kit.md .cursorrules \
  && ln -sf portable-spec-kit.md .windsurfrules \
  && ln -sf portable-spec-kit.md .clinerules \
  && mkdir -p .github && ln -sf ../portable-spec-kit.md .github/copilot-
instructions.md
```

> **Zero install.** One command. No package managers. No dependencies. Works with every AI agent. Personalized from the first session.

Portable Spec Kit

Thank You

Author: Dr. Aqib Mumtaz, Ph.D.

Specialization: Computer Science — Artificial Intelligence

Research: Multimodal AI, Healthcare AI, Autonomous Surveillance

Email: aqib.mumtaz@gmail.com

LinkedIn: linkedin.com/in/aqibmumtaz

GitHub: github.com/aqibmumtaz

v0.2.1 • April 2026